

AI ASSISTED CODING

LAB 8.2

SATHWIK VEMULAWADA

2403A53002

24BTCAICYB01-2-1

Task Description#1:

Use AI to generate test cases for a function `is_prime(n)` and then implement the function.

Requirements:

- Only integers > 1 can be prime.
- Check edge cases: 0, 1, 2, negative numbers, and large primes.

PROMPT:

V Generate test cases for a function `is_prime(n)` and then implement the function with requirements only integers > 1 can be prime and Check edge cases: 0, 1, 2, negative numbers, and large primes and provide the output which checks the working prime checker that passes all generated tests using edge coverage.

CODE:

```
[14] test_cases = [
    (0, False), # Edge case: 0 (not prime)
    (1, False), # Edge case: 1 (not prime)
    (2, True), # Edge case: 2 (prime)
    (-5, False), # Edge case: negative number (not prime)
    (999983, True), # Edge case: large prime number
    (1000000, False), # Edge case: large non-prime number
    (3, True), # Non-edge case: small prime
    (7, True), # Non-edge case: small prime
    (11, True), # Non-edge case: small prime
    (4, False), # Non-edge case: small non-prime
    (9, False), # Non-edge case: small non-prime
    (15, False) # Non-edge case: small non-prime
]
```

```
[15] import math

def is_prime(n):
    """Checks if a number is prime (greater than 1).

    Args:
        n: An integer.

    Returns:
        True if n is a prime number (greater than 1), False otherwise.
    """
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True
```

```
[16] for number, expected_output in test_cases:
    actual_output = is_prime(number)
    print(f"Input: {number}, Expected: {expected_output}, Actual: {actual_output}")

Input: 0, Expected: False, Actual: False
Input: 1, Expected: False, Actual: False
Input: 2, Expected: True, Actual: True
Input: -5, Expected: False, Actual: False
Input: 999983, Expected: True, Actual: True
Input: 1000000, Expected: False, Actual: False
Input: 3, Expected: True, Actual: True
Input: 7, Expected: True, Actual: True
Input: 11, Expected: True, Actual: True
Input: 4, Expected: False, Actual: False
Input: 9, Expected: False, Actual: False
Input: 15, Expected: False, Actual: False
```

EXPLANATION:

✦ Happy to help! I will generate a plan to create test cases, implement a prime checking function, and then test the function with the generated test cases.

- **Generate test cases:** Create a list of test cases for the `is_prime(n)` function, including edge cases like 0, 1, 2, negative numbers, and large primes, covering both prime and non-prime numbers.
- **Implement `is_prime(n)` function:** Write the Python function `is_prime(n)` that takes an integer `n` as input and returns `True` if `n` is a prime number (greater than 1) and `False` otherwise.
- **Run tests:** Iterate through the generated test cases and call the `is_prime(n)` function for each test case, printing the input and the expected and actual output.
- **Finish task:** Summarize the results of the tests and confirm if the `is_prime(n)` function works correctly based on the generated test cases.

SUMMARY:

Data Analysis Key Findings

- A list of test cases was successfully generated, covering various scenarios including edge cases (0, 1, 2, negative numbers, large prime and non-prime numbers) and non-edge cases.
- The `is_prime(n)` function was implemented in Python, correctly identifying prime numbers as integers greater than 1. The implementation efficiently checks for primality by handling base cases and iterating through potential odd divisors up to the square root of the input number.
- Running the generated test cases against the implemented `is_prime(n)` function showed that all test cases passed, with the actual output matching the expected output for every input.

Insights or Next Steps

- The comprehensive test suite, including edge cases, provides good confidence in the correctness of the `is_prime(n)` function.
- For even larger prime number checks, consider exploring more advanced primality testing algorithms like the Miller-Rabin test for improved performance.

Task Description#2 (Loops):

- Ask AI to generate test cases for `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)`.

Requirements

- Validate known pairs: $0^{\circ}\text{C} = 32^{\circ}\text{F}$, $100^{\circ}\text{C} = 212^{\circ}\text{F}$.
- Include decimals and invalid inputs like strings or None.

PROMPT:

V Generate test cases for `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)` use • Validate known pairs: $0^{\circ}\text{C} = 32^{\circ}\text{F}$, $100^{\circ}\text{C} = 212^{\circ}\text{F}$. • Include decimals and invalid inputs like strings or None Generate the output using Dual conversion functions with complete test coverage and safe type handling..

CODE:

```
[1]
✓ 0s
celsius_test_cases = [
    (0, 32.0),
    (100, 212.0),
    (25.5, 77.9),
    ("abc", None), # Assuming None as expected output for invalid input
    (None, None)  # Assuming None as expected output for invalid input
]

fahrenheit_test_cases = [
    (32, 0.0),
    (212, 100.0),
    (77.9, 25.5),
    ("xyz", None), # Assuming None as expected output for invalid input
    (None, None)  # Assuming None as expected output for invalid input
]
```

[2]
✓ 0s

```
def celsius_to_fahrenheit(c):  
    """Converts Celsius to Fahrenheit with type handling.  
  
    Args:  
        c: The temperature in Celsius.  
  
    Returns:  
        The temperature in Fahrenheit if input is valid number, None otherwise.  
    """  
    if not isinstance(c, (int, float)):  
        return None  
    return (c * 9/5) + 32  
  
def fahrenheit_to_celsius(f):  
    """Converts Fahrenheit to Celsius with type handling.  
  
    Args:  
        f: The temperature in Fahrenheit.  
  
    Returns:  
        The temperature in Celsius if input is valid number, None otherwise.  
    """  
    if not isinstance(f, (int, float)):  
        return None  
    return (f - 32) * 5/9
```

[3]
✓ 0s

```
print("Celsius to Fahrenheit Tests:")  
for celsius, expected_fahrenheit in celsius_test_cases:  
    actual_fahrenheit = celsius_to_fahrenheit(celsius)  
    print(f"Input: {celsius}°C, Expected: {expected_fahrenheit}°F, Actual: {actual_fahrenheit}°F")  
  
print("\nFahrenheit to Celsius Tests:")  
for fahrenheit, expected_celsius in fahrenheit_test_cases:  
    actual_celsius = fahrenheit_to_celsius(fahrenheit)  
    print(f"Input: {fahrenheit}°F, Expected: {expected_celsius}°C, Actual: {actual_celsius}°C")
```



```
Celsius to Fahrenheit Tests:  
Input: 0°C, Expected: 32.0°F, Actual: 32.0°F  
Input: 100°C, Expected: 212.0°F, Actual: 212.0°F  
Input: 25.5°C, Expected: 77.9°F, Actual: 77.9°F  
Input: abc°C, Expected: None°F, Actual: None°F  
Input: None°C, Expected: None°F, Actual: None°F  
  
Fahrenheit to Celsius Tests:  
Input: 32°F, Expected: 0.0°C, Actual: 0.0°C  
Input: 212°F, Expected: 100.0°C, Actual: 100.0°C  
Input: 77.9°F, Expected: 25.5°C, Actual: 25.500000000000004°C  
Input: xyz°F, Expected: None°C, Actual: None°C  
Input: None°F, Expected: None°C, Actual: None°C
```

EXPLANATION:

✦ Certainly! I will generate a plan to create test cases for both Celsius to Fahrenheit and Fahrenheit to Celsius conversion functions, implement these functions with type handling, and then run the tests to ensure complete coverage.

- **Generate test cases for temperature conversion:** Create comprehensive test cases for both `celsius_to_fahrenheit` and `fahrenheit_to_celsius` functions. These should include known conversion pairs (0°C to 32°F, 100°C to 212°F), decimal values, and invalid inputs such as strings or `None`.
- **Implement `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)` functions:** Write the Python functions `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)` to perform the respective temperature conversions. Ensure that these functions include robust type handling to gracefully manage non-numeric or invalid inputs.
- **Run tests:** Iterate through the generated test cases for both functions, call the respective conversion function for each input, and print the input, expected output, and actual output.
- **Finish task:** Summarize the results of the tests and confirm that the implemented functions correctly handle the specified test cases, including type handling and known conversions.

SUMMARY:

Data Analysis Key Findings

- The `celsius_to_fahrenheit` function correctly converts 0°C to 32°F, 100°C to 212°F, and 25.5°C to 77.9°F.
- The `celsius_to_fahrenheit` function correctly handles invalid inputs such as strings and `None` by returning `None`.
- The `fahrenheit_to_celsius` function correctly converts 32°F to 0.0°C and 212°F to 100.0°C.
- When converting 77.9°F to Celsius, a minor floating-point discrepancy was observed, resulting in 25.500000000000004°C instead of exactly 25.5°C.
- The `fahrenheit_to_celsius` function correctly handles invalid inputs such as strings and `None` by returning `None`.

Insights or Next Steps

- The temperature conversion functions demonstrate robust type handling for non-numeric inputs.
- While the tests covered known pairs, decimals, and invalid inputs, future improvements could involve adding tests for extremely large or small numeric values and considering potential floating-point precision requirements.

Task Description#3:

Use AI to write test cases for a function `count_words(text)` that returns the number of words in a sentence.

Requirement

Handle normal text, multiple spaces, punctuation, and empty strings

PROMPT:

V Generate test cases for a function `count_words(text)` that returns the number of words in a sentence. Use Handle normal text, multiple spaces, punctuation, and empty strings. Give an output with a Accurate word count with robust test case validation.

CODE:

```
[4]
✓ 0s
word_count_test_cases = [
    ("This is a normal sentence.", 5), # Normal text
    ("This sentence has multiple spaces.", 5), # Multiple spaces
    ("This sentence has punctuation, like commas!", 6), # Punctuation
    ("", 0), # Empty string
    (" ", 0), # String with only spaces
    ("  Leading and trailing spaces.  ", 4) # Leading and trailing spaces
]
```

[5]
✓ Os

```
import re

def count_words(text):
    """Counts the number of words in a string, handling multiple spaces and punctuation.

    Args:
        text: The input string.

    Returns:
        The number of words in the string, or 0 for invalid/empty input.
    """
    if not isinstance(text, str) or not text.strip():
        return 0
    # Replace punctuation with spaces and then split by spaces
    cleaned_text = re.sub(r'^\w\s', '', text)
    words = cleaned_text.split()
    return len(words)
```

[7]
✓ Os

```
print("Word Count Tests:")
for input_string, expected_count in word_count_test_cases:
    actual_count = count_words(input_string)
    print(f"Input: \"{input_string}\", Expected: {expected_count}, Actual: {actual_count}")
```

```
Word Count Tests:
Input: "This is a normal sentence.", Expected: 5, Actual: 5
Input: "This sentence has multiple spaces.", Expected: 5, Actual: 5
Input: "This sentence has punctuation, like commas!", Expected: 6, Actual: 6
Input: "", Expected: 0, Actual: 0
Input: " ", Expected: 0, Actual: 0
Input: " Leading and trailing spaces. ", Expected: 4, Actual: 4
```

+ Code

+ Text

EXPLANATION:

✦ Okay, I will generate a plan to create test cases for a `count_words` function, implement the function to handle various text inputs, and then test it thoroughly.

- **Generate test cases for word counting:** Create a list of test cases for the `count_words(text)` function. Include examples of normal text, text with multiple spaces between words, text with punctuation, and empty strings. For each test case, specify the input string and the expected word count.
- **Implement `count_words(text)` function:** Write the Python function `count_words(text)` that takes a string as input and returns the number of words in the string. The implementation should correctly handle multiple spaces and punctuation.
- **Run tests:** Iterate through the generated test cases, call the `count_words(text)` function for each input string, and print the input string, the expected word count, and the actual word count.
- **Finish task:** Summarize the results of the tests and confirm if the `count_words(text)` function accurately counts words for the specified test cases, including those with multiple spaces, punctuation, and empty strings.

SUMMARY:

Summary:

Data Analysis Key Findings

- A list of test cases was successfully generated, covering various scenarios including normal text, multiple spaces, punctuation, empty strings, strings with only spaces, and strings with leading/trailing spaces.
- A Python function `count_words(text)` was implemented to count words, correctly handling multiple spaces and punctuation.
- The `count_words` function successfully passed all the generated test cases, accurately calculating the word count for each input string.

Insights or Next Steps

- The current implementation uses regular expressions to remove punctuation. Consider alternative methods for handling punctuation if performance becomes a critical factor for very large texts.
- Expand the test cases to include more complex scenarios, such as strings with numbers, special characters other than standard punctuation, and different languages.

Task Description#4:

- Generate test cases for a BankAccount class with:

Methods:

`deposit(amount)`

`withdraw(amount)`

`check_balance()`

Requirements:

- Negative deposits/withdrawals should raise an error.
- Cannot withdraw more than balance.

PROMPT:

- V Generate test cases for a BankAccount class with: Methods: `deposit(amount)` `withdraw(amount)` `check_balance()` with requirements • Negative deposits/withdrawals should raise an error. • Cannot withdraw more than balance. and the expected output should be • AI-generated test suite with a robust class that handles all test cases.

CODE:

[8]
✓ Os

```
bank_account_test_cases = [  
    (100, 'deposit', 50, 150), # Successful deposit  
    (100, 'withdraw', 50, 50), # Successful withdrawal  
    (100, 'deposit', -50, 'ValueError'), # Negative deposit  
    (100, 'withdraw', -50, 'ValueError'), # Negative withdrawal  
    (100, 'withdraw', 150, 'ValueError'), # Withdraw more than balance  
    (200, 'check_balance', None, 200), # Check balance  
    (50, 'deposit', 100, 150), # Sequence: deposit  
    (150, 'withdraw', 75, 75), # Sequence: withdraw  
    (75, 'check_balance', None, 75) # Sequence: check balance  
]
```

[9]
✓ Os

```
class BankAccount:  
    """A simple bank account class with deposit, withdraw, and check_balance methods."""  
  
    def __init__(self, initial_balance=0):  
        """Initializes the bank account with an optional initial balance."""  
        self.balance = initial_balance  
  
    def deposit(self, amount):  
        """Deposits a specified amount into the account."""  
        if amount < 0:  
            raise ValueError("Deposit amount cannot be negative.")  
        self.balance += amount  
  
    def withdraw(self, amount):  
        """Withdraws a specified amount from the account."""  
        if amount < 0:  
            raise ValueError("Withdrawal amount cannot be negative.")  
        if amount > self.balance:  
            raise ValueError("Insufficient funds.")  
        self.balance -= amount  
  
    def check_balance(self):  
        """Returns the current balance of the account."""  
        return self.balance
```

```

[10]
✓ Os
print("Bank Account Test Results:")
for initial_balance, method_name, amount, expected_outcome in bank_account_test_cases:
    account = BankAccount(initial_balance)
    actual_outcome = None
    test_passed = False

    try:
        if method_name == 'deposit':
            account.deposit(amount)
            actual_outcome = account.check_balance()
        elif method_name == 'withdraw':
            account.withdraw(amount)
            actual_outcome = account.check_balance()
        elif method_name == 'check_balance':
            actual_outcome = account.check_balance()
        else:
            actual_outcome = f"Unknown method: {method_name}"

        if actual_outcome == expected_outcome:
            test_passed = True

    except ValueError as e:
        actual_outcome = 'ValueError'
        if expected_outcome == 'ValueError':
            test_passed = True
        else:
            test_passed = False

    print(f"Initial Balance: {initial_balance}, Method: {method_name}, Amount: {amount}, Expected: {expected_outcome}, Actual: {actual_outcome}, Test Passed: {test_passed}")

```

EXPLANATION:

- ◆ Absolutely! I will generate a plan to create test cases for a `BankAccount` class, implement the class with the specified requirements, and then run the tests to ensure the class functions as expected.
 - Generate test cases for bankaccount:** Create a list of test cases for the `BankAccount` class methods (`deposit`, `withdraw`, `check_balance`). Include test cases for successful deposits and withdrawals, attempting to deposit/withdraw negative amounts, and attempting to withdraw more than the current balance. For each test case, specify the initial balance, the method to call, the amount (if applicable), and the expected outcome (e.g., new balance, raised error).
 - Implement bankaccount class:** Write the Python `BankAccount` class with an `__init__` method to set the initial balance, and implement the `deposit`, `withdraw`, and `check_balance` methods with the specified requirements (handling negative amounts and insufficient funds).
 - Run tests:** Iterate through the generated test cases, create a `BankAccount` instance with the initial balance, call the specified method with the given amount, and verify the actual outcome against the expected outcome. Print the test case details and the result (pass/fail).
 - Finish task:** Summarize the test results and confirm if the `BankAccount` class implementation meets all the requirements based on the test cases.

SUMMARY:

Summary:

Data Analysis Key Findings

- A list of test cases was successfully created for the `BankAccount` class, covering successful deposits/withdrawals, negative amounts, insufficient funds, and checking the balance.
- The `BankAccount` class was implemented with `deposit`, `withdraw`, and `check_balance` methods, correctly handling negative amounts and insufficient funds by raising `ValueError`.
- All generated test cases passed when executed against the implemented `BankAccount` class.

Insights or Next Steps

- The current test suite provides good coverage for the specified requirements. Consider adding edge cases like depositing/withdrawing zero.
- The test results confirm that the `BankAccount` class implementation adheres to the given requirements for handling valid and invalid operations.

Task Description#5:

Generate test cases for `is_number_palindrome(num)`, which checks if an integer reads the same backward.

Examples:

121 → True

123 → False

0, negative numbers → handled gracefully.

PROMPT:

- V Generate test cases for `is_number_palindrome(num)`, which checks if an integer reads the same backward. By using of some examples like 121 → True 123 → False 0, negative numbers → handled gracefully generate an output which is a Number-based palindrome checker function validated against test cases..

CODE:

[11]

✓ Os

```
palindrome_test_cases = [
    (121, True),      # Positive palindrome
    (123, False),     # Positive non-palindrome
    (0, True),        # Zero (considered a palindrome)
    (-121, False),    # Negative number (not a palindrome)
    (1221, True),     # Another positive palindrome
    (10, False),      # Another positive non-palindrome
    (5, True),        # Single digit number (palindrome)
    (-5, False)       # Single digit negative number
]
```

Reasoning: Implement the `is_number_palindrome` function as specified.

[12]

✓ Os

```
def is_number_palindrome(num):
    """Checks if an integer reads the same backward.

    Args:
        num: An integer.

    Returns:
        True if the number is a palindrome, False otherwise.
    """
    if num < 0:
        return False
    num_str = str(num)
    return num_str == num_str[::-1]
```

[13]

✓ Os

```
print("Number Palindrome Test Results:")
for number, expected_output in palindrome_test_cases:
    actual_output = is_number_palindrome(number)
    print(f"Input: {number}, Expected: {expected_output}, Actual: {actual_output}")
```

```
Number Palindrome Test Results:
Input: 121, Expected: True, Actual: True
Input: 123, Expected: False, Actual: False
Input: 0, Expected: True, Actual: True
Input: -121, Expected: False, Actual: False
Input: 1221, Expected: True, Actual: True
Input: 10, Expected: False, Actual: False
Input: 5, Expected: True, Actual: True
Input: -5, Expected: False, Actual: False
```

+ Code

+ Text

EXPLANATION:

✦ I can help you with that! I will generate a plan to create test cases for a function that checks if a number is a palindrome, implement the function to handle different inputs including edge cases, and then run the tests.

- **Generate test cases for `is number palindrome`:** Create a list of test cases for the `is_number_palindrome(num)` function. Include examples of positive palindromes, non-palindromes, zero, and negative numbers. For each test case, specify the input number and the expected boolean output.
- **Implement `is number palindrome(num)` function:** Write the Python function `is_number_palindrome(num)` that takes an integer as input and returns `True` if it's a palindrome (reads the same backward and forward), and `False` otherwise. The function should handle edge cases like 0 and negative numbers gracefully.
- **Run tests:** Iterate through the generated test cases, call the `is_number_palindrome(num)` function for each input number, and print the input number, the expected output, and the actual output.
- **Finish task:** Summarize the test results and confirm if the `is_number_palindrome(num)` function correctly identifies palindromic numbers and handles edge cases as expected.

SUMMARY:

Summary:

Data Analysis Key Findings

- A list of test cases was created, including positive palindromes (121, 1221, 5), positive non-palindromes (123, 10), zero (0), and negative numbers (-121, -5).
- The `is_number_palindrome` function was implemented to check if an integer is a palindrome by converting it to a string and comparing it to its reverse. It correctly handles negative numbers by returning `False`.
- All defined test cases were executed against the `is_number_palindrome` function, and the actual outputs matched the expected outputs, indicating the function works correctly for the provided test cases.

Insights or Next Steps

- The current implementation converts the number to a string, which is simple and readable. For very large integers, consider an arithmetic approach to avoid potential string conversion limitations.

